LESSON 02 · LOGGING · 50 MIN

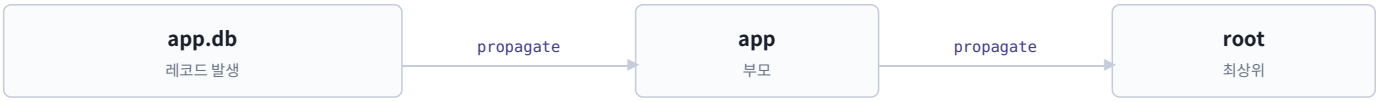
2교시 Logging 활용 — 계층·설정·운영

로거는 점(.)으로 이어진 가족입니다. 레코드는 자기 핸들러를 거친 뒤 부모에게 계속 올라갑니다. 이 성질이 중복 출력의 원인이자, 설정을 한 곳에 모을 수 있는 이유이기도 합니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

레코드는 부모에게로 올라간다



> 레벨 검사는 **맨 처음 로거에서 딱 한 번**만 합니다. 올라가는 동안 부모의 레벨은 검사하지 않고, 대신 **만나는 핸들러마다 그 핸들러의 레벨**을 검사합니다. 이 규칙 하나로 중복 출력이 전부 설명됩니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`getLogger("app.db")`	점으로 부모 자식이 결정된다	이름만 잘 지으면 계층이 생긴다
`level = NOTSET`	부모에게서 레벨을 물려받는다	자식 로거의 기본값
`getEffectiveLevel()`	실제로 적용되는 레벨을 확인	헷갈릴 때 바로 찍어 본다
`logger.propagate = False`	부모에게 올려보내지 않는다	중복 출력을 끊을 때
`dictConfig(설정딕셔너리)`	설정을 코드 밖으로 분리	환경별로 다른 설정을 줄 때
`logger.exception("...")`	except 안에서 traceback까지 기록	예외 처리에서는 항상 이것
`RotatingFileHandler`	크기가 차면 파일을 굴린다	디스크를 지키면서 파일로 남길 때
`getLogger("urllib3").setLevel(...)`	남의 라이브러리 로그를 잠재운다	콘솔이 남의 로그로 덮일 때

FILES

실습용 파일

log_config.py

```
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {
        "default": {
            "format": "%(asctime)s %(levelname)-8s %(name)s "
                      "%(filename)s:%(lineno)d %(message)s"
        }
    },
    "handlers": {
        "console": {
            "class": "logging.StreamHandler",
            "level": "INFO",
            "formatter": "default",
        },
        "file": {
            "class": "logging.handlers.RotatingFileHandler",
            "level": "DEBUG",
            "formatter": "default",
            "filename": "app.log",
            "maxBytes": 1_048_576,
            "backupCount": 3,
            "encoding": "utf-8",
        },
    },
    "loggers": {
        "app": {"level": "DEBUG", "handlers": ["console", "file"], "propagate": False},
        "urllib3": {"level": "WARNING"},
    },
    "root": {"level": "WARNING", "handlers": ["console"]},
}
```

HANDS-ON

실습 순서

STEP 01 계층 만들고 유효 레벨 확인

이름만으로 부모 자식이 생기는 것을 확인합니다.

```
# tree_demo.py
import logging
logging.basicConfig(level=logging.WARNING, format="%(name)s %(levelname)s %(message)s")

app = logging.getLogger("app")
db = logging.getLogger("app.db")

print(db.parent.name)
print(logging.getLevelName(db.getEffectiveLevel()))
app.setLevel(logging.DEBUG)
print(logging.getLevelName(db.getEffectiveLevel()))

python tree_demo.py
```

확인 포인트

- [ ] db.parent.name 이 app 으로 나온다 — 만든 적 없는 부모가 자동으로 연결된다
- [ ] 처음에는 WARNING, app 을 DEBUG로 바꾸면 db 도 DEBUG가 된다

> 자식 로거의 레벨은 기본이 **NOTSET** 이라, 가장 가까운 조상의 레벨을 그대로 씁니다. 부모 하나만 바꾸면 그 아래가 전부 따라옵니다.

STEP 02 중복 출력 재현하기 — 오늘의 핵심 (1)

같은 메시지가 두 번 찍히는 상황을 일부러 만듭니다.

```
# dup_demo.py
import logging

root_h = logging.StreamHandler()
root_h.setFormatter(logging.Formatter("[root핸들러] %(name)s %(message)s"))
logging.getLogger().addHandler(root_h)
logging.getLogger().setLevel(logging.DEBUG)

db = logging.getLogger("app.db")
db_h = logging.StreamHandler()
db_h.setFormatter(logging.Formatter("[db핸들러] %(name)s %(message)s"))
db.addHandler(db_h)

db.warning("쿼리 실패")

python dup_demo.py
```

확인 포인트

- [ ] 같은 메시지가 두 줄 나온다
- [ ] 한 줄은 [db핸들러], 한 줄은 [root핸들러] 로 표시된다

> 레코드가 app.db 의 핸들러에서 출력된 뒤, **부모를 거쳐 root 까지 올라가** 거기서도 출력된 것입니다. 중복 출력의 정체는 거의 항상 이것입니다.

STEP 03 중복을 끊는 두 가지 방법

어느 쪽을 택할지 판단 기준을 익힙니다.

```
# 방법 A · 올려보내지 않기
db.propagate = False

# 방법 B · 자식의 핸들러를 떼고 root 에만 두기
db.removeHandler(db_h)

# 각각 적용해 다시 실행
python dup_demo.py
```

확인 포인트

- [ ] A: [db핸들러] 한 줄만 나온다
- [ ] B: [root핸들러] 한 줄만 나온다

> 실무에서는 **B가 기본**입니다. 핸들러는 진입점 한 곳에만 두고, 나머지 모듈은 이름만 나눠 쓰는 것이 관리하기 쉽습니다.
> A(propagate=False)는 특정 로그만 따로 빼서 다른 파일에 남길 때처럼 이유가 분명할 때만 쓰세요.

STEP 04 dictConfig 로 설정 분리

설정을 코드에서 떼어 냅니다.

```
# 위 '실습용 파일'의 log_config.py 를 만들고
# main.py 를 아래처럼 바꿉니다
import logging.config
from log_config import LOGGING

logging.config.dictConfig(LOGGING)
logger = logging.getLogger("app.orders")
logger.info("주문 처리 시작")
logger.debug("상세 값")

python main.py && cat app.log
```

확인 포인트

- [ ] 콘솔에는 info 만, app.log 에는 debug 까지 남는다
- [ ] app.orders 는 설정에 없는데도 app 설정을 물려받아 동작한다

> **disable_existing_loggers 는 반드시 False** 로 두세요. True(기본값)면 이 설정 전에 만들어진 로거들이 조용히 꺼져서, 라이브러리 로그가 통째로 사라지는 일이 생깁니다.

STEP 05 예외를 제대로 기록하기

traceback 을 로그에 남깁니다.

```
# main.py 에 추가
try:
    1 / 0
except ZeroDivisionError:
    logger.error("계산 실패")           # 나쁜 예
    logger.exception("계산 실패")       # 좋은 예

python main.py
```

확인 포인트

- [ ] error 는 한 줄만 남는다 — 어디서 났는지 알 수 없다
- [ ] exception 은 Traceback 전체가 함께 남는다

> **except 안에서는 logger.exception()** 이 기본입니다. error 로 남기고 싶다면 **logger.error("...", exc_info=True)** 로도 같은 효과를 낼 수 있습니다.

STEP 06 로테이션 동작 확인 — 오늘의 핵심 (2)

파일이 굴러가고 오래된 것이 지워지는 것을 봅니다.

```
# rotate_demo.py
import logging, logging.handlers

logger = logging.getLogger("rot")
logger.setLevel(logging.DEBUG)
h = logging.handlers.RotatingFileHandler(
    "rot.log", maxBytes=2000, backupCount=2, encoding="utf-8")
logger.addHandler(h)

for i in range(200):
    logger.info("%04d 번째 줄 - 로테이션 실습용 메시지입니다", i)

python rotate_demo.py && ls -l rot.log*
```

확인 포인트

- [ ] rot.log, rot.log.1, rot.log.2 세 개가 생긴다
- [ ] rot.log.3 은 없다 — backupCount=2 이므로 그 이상은 삭제된다
- [ ] 각 파일 크기가 maxBytes(2000) 를 넘지 않는다

> 보관 상한 = **maxBytes × (backupCount + 1)** 입니다. 위 설정이면 최대 6KB만 남고 그보다 오래된 기록은 사라집니다.
> 더 길게 보관해야 하면 backupCount 를 늘리거나, 날짜 기준인 **TimedRotatingFileHandler** 를 쓰세요.

STEP 07 남의 라이브러리 로그 잠재우기

내 로그가 파묻히지 않게 합니다.

```
# 라이브러리가 쏟아내는 상황 재현
logging.getLogger("urllib3").setLevel(logging.WARNING)
logging.getLogger("asyncio").setLevel(logging.WARNING)

# 어떤 로거들이 살아 있는지 확인
python -c "import logging; print(sorted(logging.root.manager.loggerDict))"
```

확인 포인트

- [ ] 로거 이름 목록이 출력된다
- [ ] 레벨을 올린 뒤 콘솔이 조용해진다

> 루트 레벨을 DEBUG로 열면 **남이 만든 라이브러리 로그까지 전부** 쏟아집니다. 내 앱 로거만 DEBUG로 두고 루트는 WARNING에 두는 것이 기본 구성인 이유입니다.

STEP 08 정리하고 운영 설정 만들기

개발용과 운영용 두 벌을 준비합니다.

```
# log_config.py 에 두 벌을 두고 골라 쓰기
import os
LOGGING = DEV if os.getenv("ENV") == "dev" else PROD

ENV=dev python main.py
python main.py
```

확인 포인트

- [ ] ENV 값에 따라 콘솔 출력량이 달라진다
- [ ] 코드는 전혀 바뀌지 않았다

> 설정을 코드 밖으로 뺀 진짜 이유가 이것입니다. **배포물은 그대로 두고 환경만 바꿔** 동작을 조절할 수 있습니다.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
같은 로그가 2~3번 찍힌다	경로 위 여러 핸들러가 모두 출력	핸들러를 한 곳에만 두거나 propagate=False
dictConfig 이후 라이브러리 로그가 사라짐	disable_existing_loggers 기본값 True	설정에 "disable_existing_loggers": False 추가
로테이션이 안 일어난다	같은 파일에 핸들러를 두 개 붙임	파일 하나당 핸들러 하나만 유지
윈도우에서 로테이션 시 PermissionError	다른 프로세스가 파일을 잡고 있음	실행 중인 다른 프로세스 종료 · 파일을 열어 둔 편집기 닫기
logger.error 만 남고 원인을 모르겠다	traceback 을 안 남김	except 안에서는 logger.exception() 사용

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- [ ] app.db 로거를 만들면 app 로거는 언제 만들어지나요?
- [ ] 레벨 검사는 몇 번, 어디서 일어나나요?
- [ ] 같은 메시지가 두 번 찍힐 때 확인해야 할 것 두 가지는?
- [ ] disable_existing_loggers 를 False 로 두는 이유는?
- [ ] maxBytes=1MB, backupCount=3 이면 디스크를 최대 얼마나 쓰나요?

한 줄 정리

핸들러는 진입점 한 곳에, 이름은 모듈마다. 이 원칙만 지켜도 로깅 문제의 대부분은 생기지 않습니다.

다음 시간 → 3교시 — 여기서부터 테스트입니다

WORKSHEET

막힌 STEP 번호 · 화면에 나온 오류 메시지 · 물어볼 것